

PREFLOW: Preemptive LLM Prefill Scheduling with Provable FCFS-Relative Protection

Long-context inference turns prompt processing into a scheduling problem. A single prefill worker may see interactive prompts alongside retrieval-augmented requests with tens of thousands of tokens and the occasional 100K-token document or code context. Their service requirements differ by orders of magnitude, and they often arrive in bursts.

The scheduling problem is especially visible in prefill/decode (PD) disaggregated serving. Prefill and decode run on separate workers, so the prefill scheduler can focus on time to first token (TTFT) and the rate at which it hands completed KV state to decode. At every scheduling boundary, it must choose which prompt gets the next unit of prefill compute.

FCFS makes that decision predictable and creates head-of-line blocking: one 100K-token prompt can hold dozens of short prompts behind it. Shortest-job-first (SJF) and shortest-remaining-processing-time (SRPT) break that convoy and reduce mean latency by letting short work overtake long work. Neither policy limits the delay that those overtakes can accumulate on a particular request.

PREFLOW is, to our knowledge, the first preemptive chunked-prefill scheduler to combine aggressive work-aware reordering with a provable per-request bound on modeled completion inflation relative to FCFS. Each request receives a fixed entitlement based on the service available at its arrival. PREFLOW can reorder the queue freely within the set of actions that preserve every entitlement. Under the stated cost and admission assumptions, later arrivals cannot weaken the bound.

Existing prefill schedulers usually add aging to a shortest-work score and rely on the resulting priority to balance latency and fairness. PREFLOW keeps the latency objective intact. Starvation protection instead restricts which actions are legal, and the scheduler takes the locally optimal work-aware choice from the safe set.

At its most permissive tested setting, PREFLOW captures about 92% of SRPT’s mean-TTFT improvement while reducing the worst matched-request slowdown from $7.52\times$ to $2.37\times$. With 30% configured slack, it captures about 81% of SRPT’s improvement and keeps the observed maximum at $1.37\times$. The measured operating points follow a different benefit-harm Pareto curve from aging-based preemptive scheduling.

In an aging scheduler, predicted work and accumulated waiting time share one score, so changing the aging coefficient changes both the latency objective and the amount of protection. That coefficient has no direct service meaning: it does not tell an operator whether a request may finish 20%, 200%, or 2,000% later than under FCFS. Its effect also changes with load, prompt-size mix, chunk size, and hardware.

PREFLOW’s fixed entitlement separates inherited congestion from delay caused by later reordering. At each chunk boundary, the scheduler chooses the highest-value action that preserves all existing entitlements.

The operator sets one quantity with direct semantics: the maximum modeled inflation over a request’s arrival-state FCFS baseline. The scheduler uses the remaining freedom to reorder work. Its latency score contains no aging term or workload-specific fairness coefficient.

A tighter benefit-harm Pareto frontier

The first figure captures the main result. The orange PrefillOnly points trace the tradeoff produced by tuning an aging coefficient. PREFLOW’s green points do not continue that curve. They occupy a lower-harm region that none of the tested aging settings reaches, with nearly the same mean-latency benefit. Separating optimization from protection changes the set of attainable operating points.

The horizontal axis measures mean TTFT improvement over FCFS, so farther right is better. The vertical axis is the largest matched-request TTFT divided by the same request’s TTFT under FCFS, so lower is safer. Moving along a curve changes that policy’s protection parameter. Moving from the orange curve to the green one changes how the scheduler enforces protection.

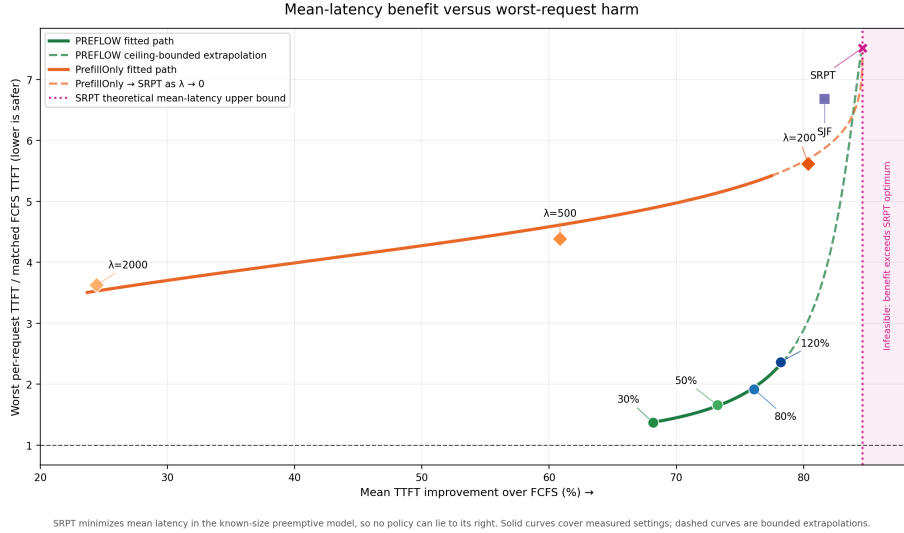


Figure 1: Mean TTFT benefit versus worst-request harm

Each policy uses request size and progress differently:

Policy	Scheduling decision	What is optimal or protected?
SJF	Start the smallest isolated prefill and run it to completion	Classic non-preemptive mean-completion rule when all jobs are available; no per-request slowdown bound
SRPT	Always run the request with least remaining work	Optimal mean flow time in the known-size preemptive single-server model; no per-request slowdown bound
PrefillOnly-style aging	Combine predicted work with a linear waiting-time offset	Soft starvation mitigation; neither the latency objective nor a per-request bound is preserved exactly
EDF ablation	Always run the request with the earliest fixed PREFLOW deadline	Tests the deadlines without PREFLOW’s greedy stretch optimization; largely collapses toward FCFS
PREFLOW	Choose the best stretch-reducing action from the set certified safe	Locally optimal choice within the feasible action set, plus a modeled FCFS-relative completion bound

The EDF ablation falls outside the plotted region. It uses the same frozen FCFS-relative deadlines as its only priority and removes PREFLOW’s greedy choice among safe requests. With 120% slack, EDF improves mean TTFT by 5.8% and reaches $1.60\times$ worst slowdown. The ablation isolates the source of PREFLOW’s gains: deadlines provide protection, while the greedy choice among safe actions provides most of the latency improvement.

SRPT is the rightmost measured point and the vertical dotted line. It is optimal for mean latency under the classic known-size preemptive model, which makes it a useful efficiency ceiling here. SRPT improves mean TTFT by 84.6%, while the worst request takes $7.52\times$ its matched FCFS TTFT. SJF reaches 81.7% mean improvement with $6.68\times$ worst-request slowdown.

The orange path shows the effect of tuning an aging coefficient. Aggressive settings stay close to shortest-work scheduling and produce both high mean benefit and high worst-request harm. Stronger aging changes the work-minimizing score at every decision. PrefillOnly with $\lambda = 200$, for example, improves mean TTFT

by 80.4% and reaches $5.62\times$ worst-request slowdown. At $\lambda = 2000$, the worst slowdown remains $3.63\times$ while mean improvement falls to 24.4%.

PREFLOW occupies a different region. With 30% slack, it improves mean TTFT by 68.2% and limits the worst observed matched slowdown to $1.37\times$. The most protective PrefillOnly setting improves mean TTFT by only 24.4% and still reaches $3.63\times$, so PREFLOW is better on both axes at these operating points. The 50% and 80% PREFLOW settings reach 73.2%/1.66 $\times$ and 76.1%/1.92 $\times$.

At 120% slack, PREFLOW retains 92% of SRPT’s measured improvement and cuts its worst slowdown by more than threefold. It also comes within 2.2 percentage points of aggressive PrefillOnly while reducing worst slowdown from $5.62\times$ to $2.37\times$. All four PREFLOW settings lie on the same low-harm path below the aging curve. Solid segments connect measured settings; dashed segments are bounded extrapolations.

An aging scheduler puts waiting time directly into the work score. Accumulated wait can therefore move a request ahead of one with higher latency value. PREFLOW instead removes any action that would violate a fixed entitlement, then selects the highest-scoring action left. Protection affects the decision only when the preferred action becomes unsafe.

If $\mathcal{A}(t)$ is the set of next-chunk actions that preserve every deadline, PREFLOW executes

$$i^*(t) = \arg \max_{i \in \mathcal{A}(t)} \frac{1}{p_i r_i(t)}.$$

At each decision boundary, this is the highest stretch-priority action in the complete safe set. The claim is local because future arrivals are unknown. The feasibility test does not alter the latency ranking, and that ranking cannot override an entitlement.

SLO-aware scheduling applies when requests already carry meaningful application deadlines. PREFLOW covers the common case where the local prefill worker knows the cost, progress, and queue state, while the application provides no deadline for each request.

Predicting the cost of a prefill chunk

The scheduler needs a common service unit for the isolated cost of a request, its remaining work after partial progress, and the exact next chunk the engine could dispatch. Token counts alone miss kernel launch overhead, attention implementation costs, parallelism, and device behavior, so wall-clock protection needs a runtime model.

PREFLOW calibrates a deterministic cost model when the prefill worker starts. Let C be the configured full chunk size and h the history before that chunk. The

worker sends calibration prefills through the same model path as real work and records the device execution time of each chunk. For a full chunk, it fits

$$T_C(h; \theta) = \theta_0 + \theta_1 z + \theta_2 z^2, \quad z = \frac{h}{H},$$

where H is a history normalization constant. The features $1, h, h^2$ capture a fixed invocation cost and the growth of attention cost with context. The fitted slopes are projected so that

$$\frac{dT_C(h; \theta)}{dh} \geq 0$$

throughout the supported context range. The model may flatten where the measurements support it, but the projection prevents quadratic extrapolation from predicting that a later chunk becomes cheaper.

PREFLOW also measures a sparse set of sizes between one token and C for prompts shorter than a full chunk and for final partial chunks. Their median costs form a positive, non-decreasing short-chunk curve $S(c)$. For $0 < c \leq C$, define the predicted non-final cost

$$T_{\text{nf}}(c, h) = S(c) + \frac{c}{C} (T_C(h) - T_C(0)).$$

This preserves the measured launch floor for small requests while scaling only the history-dependent component by the fraction of a full chunk. A separately measured non-negative finalization overhead δ_f is added only to the last chunk:

$$T(c, h, f) = T_{\text{nf}}(c, h) + f\delta_f, \quad f \in \{0, 1\}.$$

The total predicted work over an interval is the sum of the engine’s fixed chunk actions. Let $\mathcal{P}(u, v)$ be the ordered (h, c, f) triples produced by partitioning the token interval from history u to history v with the configured chunk size, including the final partial chunk. Define

$$W(u, v) = \sum_{(h, c, f) \in \mathcal{P}(u, v)} T(c, h, f).$$

If request i begins with reusable history c_i , currently has history $h_i(t)$, and completes at prompt length n_i , its isolated and remaining work are

$$p_i = W(c_i, n_i), \quad r_i(t) = W(h_i(t), n_i).$$

The next action has cost $q = T(c, h_i, f_i)$, using the exact chunk size and final-chunk status the serving engine has already determined. The scheduler keeps the configured chunk size fixed.

Calibration runs only while the worker is empty. It includes a discarded warmup, repeated full-length profiles, and short-query profiles. Once the model is fit, all calibration requests are removed, the prefix cache is reset, and the code checks that the KV blocks used for profiling have been returned. The resulting work unit is a predicted millisecond.

Turn FCFS into a per-request entitlement

PREFLOW maintains a logical service clock t in predicted-cost units. It advances only after dispatched prefill work completes. Scheduled but unfinished chunks remain outstanding work; they are not counted as completed service prematurely.

When request i arrives at logical time a_i , freeze the legal FCFS continuation from the state it sees. Let Q_i be all unfinished prefill work ahead of it in that no-future-arrivals continuation, including outstanding work exactly once. Its FCFS baseline is

$$B_i = Q_i + p_i.$$

PREFLOW converts the baseline into an immutable deadline

$$D_i = a_i + \rho B_i,$$

where $\rho \geq 1$. An operator configures the equivalent inflation: 30% slack means $\rho = 1.3$, 50% means $\rho = 1.5$, and so on.

The baseline separates inherited congestion from delay caused by reordering. A request arriving at an idle worker receives a tight deadline. The same request arriving behind a burst receives a larger one because its baseline includes the work already present. Future arrivals cannot enlarge it.

A single slack value keeps the same meaning across workloads: 30% means 30% modeled completion inflation relative to that request’s arrival-state FCFS continuation.

Optimize stretch inside the safety envelope

The deadline determines which actions are safe. A separate latency objective ranks the safe actions.

Why optimize stretch?

Let $F_i = C_i - a_i$ be request i 's flow time and p_i its predicted isolated prefill work. Mean latency minimizes $\sum_i F_i$, so one second of delay carries the same weight for every request. In the ideal preemptive single-server model, this leads to SRPT. The comparison becomes less fair when isolated service requirements differ by orders of magnitude.

Stretch normalizes latency by the work the request would need on its own:

$$S_i = \frac{F_i}{p_i}.$$

Suppose one request needs 0.1 seconds of isolated service and another needs 10 seconds. A raw-latency objective assigns the same cost to one additional second of waiting for either request. Yet the short request now takes 1.1 seconds, or $11\times$ its isolated work, while the long request takes 11 seconds, or $1.1\times$. Stretch captures this difference.

A request taking twice its isolated service has stretch two regardless of its absolute prompt length. Minimizing total stretch asks the scheduler to reduce relative waiting inflation across requests whose costs differ by orders of magnitude. Algebraically,

$$\sum_i S_i = \sum_i \frac{1}{p_i} F_i,$$

so total stretch is weighted flow time with weight $w_i = 1/p_i$.

PREFLOW uses isolated work p_i to rank safe requests and the larger arrival-state FCFS baseline B_i to decide which actions are legal. The scheduler can therefore optimize aggressively without spending more slack than the completion contract allows.

Deriving the local priority

The stretch priority follows from a pairwise interchange argument. Consider two available unfinished requests i and j , with remaining work r_i and r_j , and ignore the completion costs common to both orders. Completing i before j incurs

$$J_{i \rightarrow j} = w_i r_i + w_j (r_i + r_j),$$

whereas completing j before i incurs

$$J_{j \rightarrow i} = w_j r_j + w_i (r_j + r_i).$$

The first order is no worse exactly when

$$\frac{w_i}{r_i} \geq \frac{w_j}{r_j}.$$

Substituting $w_i = 1/p_i$ gives PREFLOW's weighted-shortest-remaining-work index:

$$\pi_i(t) = \frac{1}{p_i r_i(t)}.$$

The $1/p_i$ term values intrinsically short requests; $1/r_i$ also recognizes a partially completed request that is now cheap to finish. Without protection constraints, ordering by this index is the pairwise-optimal order for the currently released weighted-completion problem.

Now let $\mathcal{A}(t)$ contain exactly the requests whose next full chunk preserves all fixed deadlines. PREFLOW chooses

$$i^*(t) = \arg \max_{i \in \mathcal{A}(t)} \pi_i(t).$$

At each scheduling boundary, PREFLOW takes the highest-value action in the current safe set. If the preferred action is unsafe, it takes the next safe action in stretch order. Unknown future arrivals and non-preemptive chunks make this a local result. Within that scope, the choice is exact, and no workload-dependent aging weight enters the score.

The feasibility test does not depend on stretch. A deployment could rank safe actions by tenant weights, value, or another latency objective while keeping the same protection rule.

Constructing the safe action set

At each chunk boundary, order all released unfinished requests by fixed deadline,

$$D_{(1)} \leq D_{(2)} \leq \dots \leq D_{(m)},$$

with deterministic arrival-order tie-breaking. For the first k requests in that order, define residual prefix slack

$$\sigma_k(t) = D_{(k)} - t - \sum_{j=1}^k r_{(j)}(t).$$

The scheduler examines legal candidates in decreasing π_i . It dispatches the first candidate whose complete next chunk preserves every deadline. If no preferred candidate is safe, it dispatches an earliest-deadline rescue action. Remaining

work, EDF order, and prefix slack are recomputed at the next boundary; no request is permanently locked.

Exact feasibility test

Consider one work-conserving prefill resource. All unfinished requests are already released, charged work is additive, and scheduling decisions happen at chunk boundaries. A dispatched chunk runs to completion, so PREFLOW tests its full cost before dispatch. The KV-admission extension comes afterward.

Lemma 1: EDF-prefix feasibility

For released jobs on a preemptive unit-speed resource, the current state is feasible if and only if

$$\sigma_k(t) \geq 0 \quad \text{for every EDF prefix } k.$$

Proof. The first k jobs in deadline order require $R_k(t) = \sum_{j=1}^k r_{(j)}(t)$ service by $D_{(k)}$. The resource can provide only $D_{(k)} - t$ service before then. Thus $R_k(t) \leq D_{(k)} - t$, or equivalently $\sigma_k(t) \geq 0$, is necessary.

It is also sufficient. Run jobs in the deterministic EDF order and suppose the first missed deadline is $D_{(k)}$. EDF cannot execute a job later in that order while work from this prefix is pending. Missing $D_{(k)}$ would therefore mean that the prefix required more than $D_{(k)} - t$ service, contradicting $\sigma_k(t) \geq 0$. $\square$

Lemma 2: the safe-chunk criterion

In a feasible current state, suppose candidate i has EDF rank m_i , and its next non-preemptive chunk costs $q \leq r_i(t)$. Executing that chunk preserves feasibility if and only if

$$q \leq \min_{k < m_i} \sigma_k(t),$$

where the minimum of an empty set is infinity.

Proof. An EDF prefix before position m_i does not contain request i . Running i 's chunk advances the clock by q without reducing the work in that prefix, so its slack falls from σ_k to $\sigma_k - q$.

Every prefix at or after m_i contains request i . For those prefixes, time advances by q and remaining work also falls by q , leaving slack unchanged. The candidate is therefore safe exactly when every earlier prefix has at least q slack. $\square$

The earliest-deadline request has no earlier prefix, so its next chunk is always safe while the state remains feasible. It is the rescue action when no preferred

candidate passes the test. A later-deadline request may consume only the minimum slack of the prefixes it bypasses; this is the exact overtaking budget.

The FCFS-relative completion bound

The full bound follows by induction over arrivals and chunk completions under five assumptions:

1. Every realized action takes no more time than the cost charged to it.
2. PREFLOW tests the entire non-preemptive chunk before dispatch.
3. B_i upper-bounds a legal no-future-arrivals FCFS continuation that also preserves all older deadlines.
4. Every admitted action passes the exact prefix test, or a sound conservative admission test.
5. The scheduler continues making progress while unfinished work exists.

Assume the state before request i arrives is feasible. By definition of B_i , there is a legal continuation that preserves all old deadlines and completes i by $a_i + B_i$. Since $\rho \geq 1$,

$$a_i + B_i \leq a_i + \rho B_i = D_i.$$

The same continuation is therefore a feasible witness after adding i .

After an action, if the prior state was feasible and PREFLOW executes a chunk accepted by Lemma 2, every successor prefix remains feasible. Lemma 1 then supplies a continuation meeting all fixed deadlines.

The empty system is feasible. The arrival step preserves feasibility whenever a request is added, and the action step preserves it after every dispatch. Under the conservative-cost and progress assumptions, the maintained continuation is realizable. Every accepted request therefore completes by

$$C_i \leq D_i = a_i + \rho B_i, \quad \frac{C_i - a_i}{B_i} \leq \rho.$$

Because B_i and D_i are frozen, later arrivals cannot weaken this bound. They may exhaust slack and force PREFLOW to execute EDF-like rescue work. They cannot legally consume capacity already reserved for an older request.

Extending protection across KV admission

The compute proof assumes that a selected request can execute. A real serving engine also has a waiting/running boundary: a request may need a resident slot and enough KV capacity before it can run.

For a waiting request i , let $G_i(t)$ conservatively bound the work needed to drain the resident prefill set sufficiently for admission. With remaining post-admission work $r_i(t)$, its latest protection time is

$$L_i(t) = D_i - G_i(t) - r_i(t).$$

By L_i , the scheduler must stop admitting younger work that could enlarge the blocking set, drain the resident work covered by G_i , admit i as soon as its full-sequence reservation fits, and then preserve compute-prefix feasibility. If the drain finishes within G_i and execution within r_i , the request completes by D_i . Multiple critical waiting requests are protected in deadline order.

This sufficient condition is conservative and leaves the serving engine’s KV allocation and waiting/running lifecycle unchanged. It reserves enough time for a protected request to become executable without a memory-aware replay oracle.

Experimental setup

We evaluated PREFLOW on a bursty synthetic long-context workload in which queue ordering has a large effect.

- Model: Qwen3-30B-A3B-Instruct-2507 architecture with dummy weight loading.
- Topology: one TP=2 prefill instance and one TP=2 decode instance across four Ascend NPUs.
- Traffic: 1,000 requests from a Markov-modulated Poisson process averaging 0.5 requests/s over a 2,000-second arrival span.
- Prompt lengths: 4K, 8K, 16K, 32K, 65K, and 100K tokens, with shares of 30%, 25%, 20%, 14%, 8%, and 3%.
- Output: 128 requested decode tokens per request.
- Execution: 4,096-token chunks, eight resident prefill requests, exactly one scheduled prefill request per execution step, and asynchronous scheduling.
- PREFLOW cost: startup profiling of actual chunk execution; the comparison policies retain their policy-specific work ranking.
- Scope: prefix caching disabled; one fixed workload seed.

We compared FCFS, non-preemptive SJF, preemptive SRPT, PrefillOnly-style aging with $\lambda \in \{200, 500, 2000\}$, pure EDF with 120% slack as a deadline-only ablation, and PREFLOW with 30%, 50%, 80%, and 120% FCFS-relative slack. All 11 configurations completed the same 1,000 requests with no failures. Request IDs, prompt lengths, and scheduled arrivals were identical across runs.

Reordering helps, but the harm has a tail

For each request, we divide its policy TTFT by the same request’s TTFT in the FCFS run. Ratios below one are improvements; ratios above one are slowdowns.

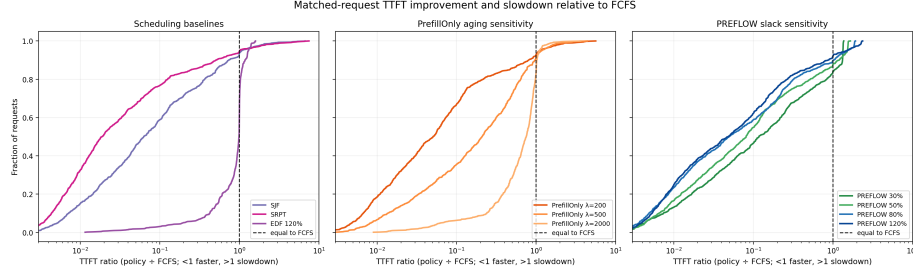


Figure 2: Matched-request TTFT ratios

SJF and SRPT accelerate most requests by letting many short prompts pass a few very long ones. Mean latency falls, while those long requests absorb the delay.

PrefillOnly moves continuously between aggressive shortest-work behavior and a more FCFS-like regime as λ changes. The required value is specific to this workload, and none of the settings assigns a known maximum slowdown to an individual request.

PREFLOW controls this tradeoff through its slack parameter. It improves 84.0% of requests at 30% slack and 92.1% at 120% slack. Aggressive PrefillOnly improves 92.9%, only 0.8 percentage points more than PREFLOW 120%, but permits $5.62\times$ rather than $2.37\times$ worst slowdown. SRPT improves 94.6% and reaches $7.52\times$. PREFLOW keeps nearly all of the broad benefit without placing most of the cost on a few long requests.

Tapering tails versus a hard ceiling

The inverse slowdown distribution shows the shape of the tail. The horizontal axis is per-request TTFT divided by matched FCFS TTFT. The vertical axis is the fraction of requests slower than that ratio, on a logarithmic scale. Both main panels use the same $1\times$ to $8\times$ horizontal range; the inset enlarges the $1\times$ to $2.5\times$ region so the four PREFLOW endpoints and configured bounds remain readable.

The left panel contains SJF, SRPT, and all measured PrefillOnly settings. Their curves taper as severe harm becomes rarer, but these policies never make further slowdown illegal. Aging bends and shortens the empirical tail without changing the support of the scheduling rule. The endpoints from $3.63\times$ to $7.52\times$ belong to this trace; another arrival sequence could move them in either direction.

PREFLOW’s curves preserve a broad region of beneficial reordering and then end abruptly near the configured FCFS-relative boundary. Once prefix slack is exhausted, the feasibility test forbids additional overtaking. SJF, SRPT, and aging only express preferences, so they can make extreme slowdown less likely

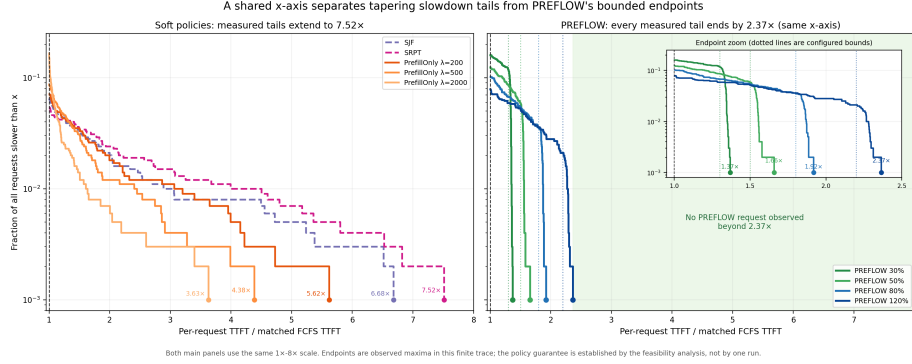


Figure 3: FCFS-relative slowdown tails

without excluding it. PREFLOW removes modeled schedules beyond the ceiling from the legal action set.

All four PREFLOW settings show the same cutoff behavior as slack moves from 30% to 120%. Slack changes the location of an explicit boundary, and the feasibility test applies the same rule as the queue changes. No aging coefficient has to be fitted to this workload.

The observed endpoints sit slightly beyond the dotted configured bounds because the plot measures wall-clock ratios from separate executions while the shield operates on fitted costs. We return to this gap below.

The queue shows how each policy responds to bursts

The queue trajectories show how each policy handles a burst. The filled area is the number of requests waiting for admission, sampled every five seconds, and the thin line is the resident running set.

FCFS develops a large, sustained waiting queue during the first burst: 52.9 requests on average and a peak of 128. Pure EDF behaves similarly because its cumulative FCFS-relative deadlines largely preserve arrival order. EDF averages 49.7 waiting requests, confirming the ablation result: deadlines alone protect requests but do little to remove the convoy.

SJF and SRPT drain request count most aggressively. SRPT averages only 6.5 waiting requests and peaks at 22. This serves short requests quickly, while the slowdown tails show the delay transferred to the remaining long requests.

PrefillOnly's queue shape depends strongly on its aging coefficient. Mean waiting occupancy rises from 9.2 requests at $\lambda = 200$, to 19.8 at $\lambda = 500$, and 39.6 at $\lambda = 2000$. The conservative setting starts to resemble FCFS, while the aggressive setting resembles shortest-work scheduling.

Across 30% to 120% slack, PREFLOW's mean waiting occupancy ranges from

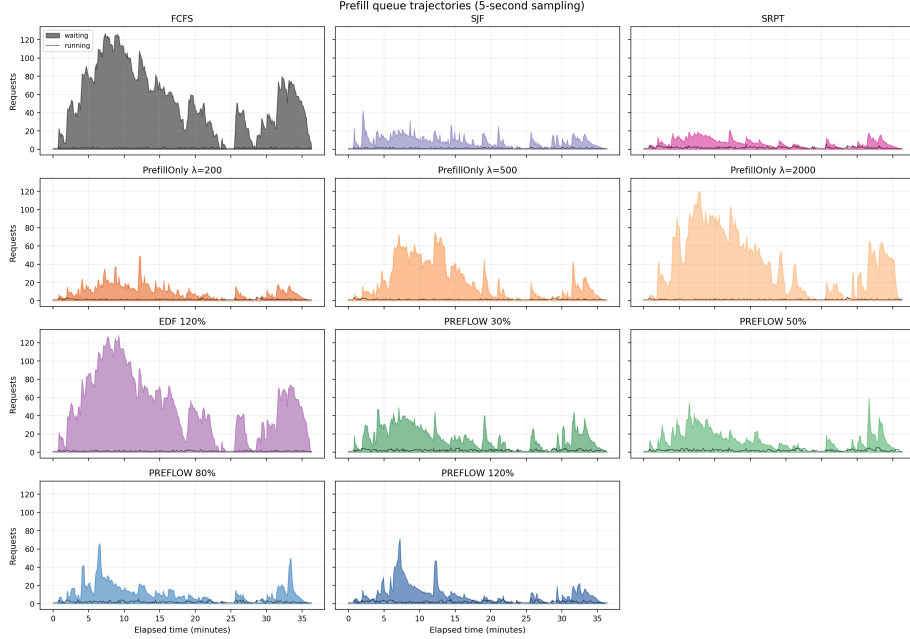


Figure 4: Prefill queue trajectories

15.0 down to 9.7 requests. Increasing slack gives the scheduler more freedom to drain short work, while fixed deadlines prevent it from achieving a small queue by abandoning requests at the back.

Queue length measures occupancy rather than fairness, and request count hides remaining work. A queue with a few 100K-token prompts may still be expensive. Together with the slowdown tails, these trajectories show that PREFLOW removes most of the FCFS convoy without SRPT’s worst-request behavior. At 120% slack, PREFLOW averages 9.7 waiting requests versus SRPT’s 6.5, while reducing the worst matched slowdown from $7.52\times$ to $2.37\times$.

Effect on end-to-end latency

PREFLOW controls the prefill scheduler, but users observe end-to-end latency. We repeated the matched-request and benefit-harm analyses using full E2E latency, including the 128-token decode.

PREFLOW spans from 66.6% to 76.7% mean E2E improvement, with worst-request E2E slowdown from $1.43\times$ to $2.30\times$. PrefillOnly $\lambda = 200$ reaches 78.8% mean improvement with $5.38\times$ worst slowdown. Measured SRPT reaches 83.4% mean improvement with $6.95\times$ worst slowdown.

The same frontier appears in end-to-end latency. PREFLOW at 120% slack

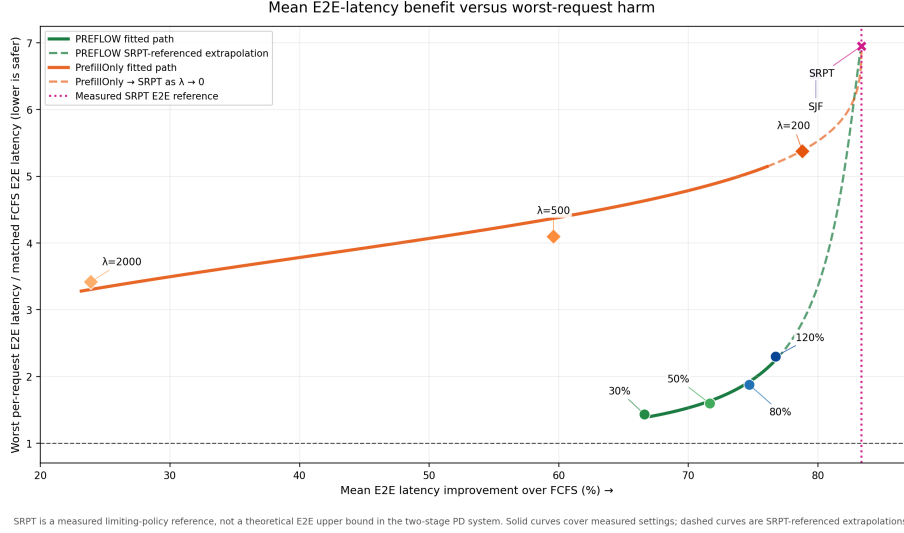


Figure 5: Mean E2E benefit versus worst-request harm

comes within 2.1 percentage points of aggressive PrefillOnly’s mean improvement and cuts worst harm from $5.38\times$ to $2.30\times$. Decode adds latency after prefill, but the separation between the policies remains.

The theorem applies only to prefill. In the two-stage PD result, SRPT is a measured reference and supplies no theoretical upper bound on full E2E latency. PREFLOW’s measured benefit-protection advantage nevertheless survives the decode stage in this experiment.

Why the wall-clock endpoints exceed the configured bounds

The proof is exact in the cost units charged by the scheduler. The current startup profiler fits median chunk times and produces point estimates without a conservative upper bound. The plotted ratios also compare separate PREFLOW and FCFS executions, so they include run-to-run noise, asynchronous overlap, memory admission, KV transfer, and cost-model residuals.

The denominator also differs. The theorem uses B_i , the predicted FCFS continuation frozen inside the PREFLOW run. The figure divides by the same request’s observed TTFT in a separate FCFS run. Even perfect enforcement in modeled units can therefore place empirical points on either side of the plotted $1 + \text{slack}$ line.

The measured worst TTFT ratios therefore slightly exceed every configured boundary: $1.369\times$ for 30% slack, $1.658\times$ for 50%, $1.920\times$ for 80%, and $2.365\times$ for 120%, versus modeled bounds of $1.3\times$, $1.5\times$, $1.8\times$, and $2.2\times$.

The measurements show a hard ceiling in modeled work and a much tighter wall-clock tail than the soft policies. The gap also measures the translation error between predicted service and separate wall-clock runs. These results do not establish an exact wall-clock guarantee.

The EDF-prefix test needs additive, conservative action costs but does not depend on the quadratic fit. A stricter runtime contract could charge an empirical upper quantile, inflate predictions by a measured error envelope, or update a safety margin online. Any of these changes would leave the deadline rule, priority policy, and feasibility proof intact. Better cost estimation tightens the mapping from the modeled ceiling to wall-clock time without changing the scheduler.

What PREFLOW adds

PREFLOW combines established techniques, including EDF, SRPT, chunking, and quadratic regression, through a new scheduling contract.

Aging-based scheduling	PREFLOW
Soft starvation mitigation	Hard modeled feasibility constraint
Workload-specific coefficient	One parameter with workload-independent semantics
Efficiency and fairness mixed in one score	Efficiency optimized subject to protection
Later arrivals may extend the harm tail	Immutable FCFS-relative deadline
Empirical tail reduction	Proven modeled completion bound

An online FCFS counterfactual creates each entitlement. The EDF-prefix certificate identifies safe chunks, and a separate efficiency objective ranks them. A conservative drain obligation carries that protection across KV admission.

Applications should use external deadlines when they have them. PREFLOW covers the local case in which the worker knows request cost, progress, and backlog but receives no meaningful deadline for each request.

Scope and next steps

This first study uses a synthetic heterogeneous workload, one seed, dummy weight loading, and no prefix caching. It tests the scheduling mechanism under a controlled workload; production claims will require more workloads and hardware platforms.

The next evaluation step is to repeat the comparison on diverse real traffic traces and platforms, including dynamic prefix reuse. The cost model also needs conservative calibration and online correction if the modeled bound is to become a deterministic wall-clock contract. Fleet-level routing, admission

control, and real application deadlines remain complementary layers outside the local scheduler.

Conclusion

Preemption reduces latency for heterogeneous LLM prefills, although repeated overtaking can make the largest requests absorb most of the delay. PREFLOW derives a fixed completion entitlement from each request’s arrival state and spends only slack that is safe for the full queue. One parameter replaces the workload-specific aging coefficient, and its meaning remains stable: FCFS defines the entitlement, while the work-aware score ranks the actions allowed by that boundary.